Module 6: Secure AI engineering

By Innovation Team

Secure AI engineering

Ship it so the CISO can sign it



Threat-model the Lab 5 agent, map it to the OWASP LLM Top 10 and compliance frameworks, and produce a control document defendable in front of a CISO - every control backed by a test.

How to use this module



Think of it like...

A simple everyday comparison so the idea sticks, even if you are new to IT.



You've seen this before

An example from normal life, outside of work.



SOC example

How a Security Operations Center (SOC) team uses it. The guide adds tips for other IT roles.



Try it

An offline HTML simulation. Click, observe, learn. No real AI and no data leaves your PC.



Lab 6

A browser simulation for every section, then the hands-on lab on your Module 0 VM (CPU only).

Your path: read the topic slides → open the simulation → do Lab 6 at the end.

Start here

1.

   This deck is your main path. Go slide by slide.

2. 

   Do each simulation

   When a slide says Try it, open that file from the simulations folder.

3. 

   Do Lab 6 at the end

   A browser simulation for every section, then the hands-on lab on your Module 0 VM (CPU only).

Choose your path

Who is it for: DevOps, security engineers and other technically literate IT staff who have completed Modules 0 and 1. No prior experience building with AI is needed. Not comfortable with code? Use the no-coding reviewer track (Track A).

Track A - Walkthrough (no terminal)

For managers, GRC, Service Desk, or anyone short on time.

1. Read the slides.
2. Do simulations 1 to 5 when a slide says Try it.
3. Do the Lab 6 walkthrough in the browser (sim6_lab6_walkthrough).
4. Take the final check.

Time: about 3 hours. Nothing to install.

Track B - Hands-on (your Module 0 VM)

For DevOps, SOC and security engineers, developers.

1. Read the slides and do the simulations.
2. Do Lab 6 on the same VM as Labs 0 to 5: Oracle Linux 9, Python 3.9. **No GPU, and no model is even needed.**
3. Generate the document; run the tests and the scan.
4. Take the final check.

Time: about 4.5 hours.

Same lesson on both tracks. Lab 6 needs no model and no new downloads - only Python 3.9 and the standard library. It reads the Lab 5 agent and produces a document about it.

What you will be able to learn

Five sections. Each ends with a simulation, a self-check and the key takeaways.

6.1 Threat-model an AI system

Map the data flow and trust boundaries, and find exactly where untrusted input enters.

6.2 OWASP LLM Top 10

Use the shared risk checklist, and map each risk to a control (or a documented N/A).

6.3 Data governance

Classify every data asset by sensitivity, PII and residency - before protecting it.

6.4 Deployment patterns

Choose a hosting pattern by data sensitivity; justify local hosting for sensitive data.

6.5 Compliance mapping

Map controls to NIST AI RMF, ISO 42001, ISO 27001 and SOC 2 - each backed by evidence.

6.6 Lab 6

Generate a Threat Model & Control Document for the Lab 5 agent, defendable in front of a CISO.

6.1 Threat-model an AI system

Before you defend it, draw it - and find the doors.

Threat-modelling: draw it, then find the doors

What it is

A **threat model** is a structured walk-through of your system that answers: **what is it, where does untrusted input get in, what could go wrong, and what stops it?** You do not need a heavy tool - you need the **data flow drawn honestly**, the **trust boundaries** marked (lines where trust changes), and a **decision for every threat**. STRIDE (Spoofing, Tampering, Repudiation, Info disclosure, Denial of service, Elevation) is just a checklist to make sure you miss none.

In other IT roles: No threat may be left undecided - "accepted, because..." is a valid decision.



Think of it like...

A **building security walk-through.** A guard walks the floor plan, marks every door and window, asks "what could someone do here?", and checks each has a lock - or a reason it is safe.



You've seen this before

Every security review of a new system starts here: a diagram, the trust boundaries, and a list of threats with decisions. AI systems are no different - they just add the model as a new untrusted actor.



In the SOC

For the Lab 5 agent, untrusted input enters in **two** places: what the **model proposes**, and the **ticket comments** it reads. The review focuses there.

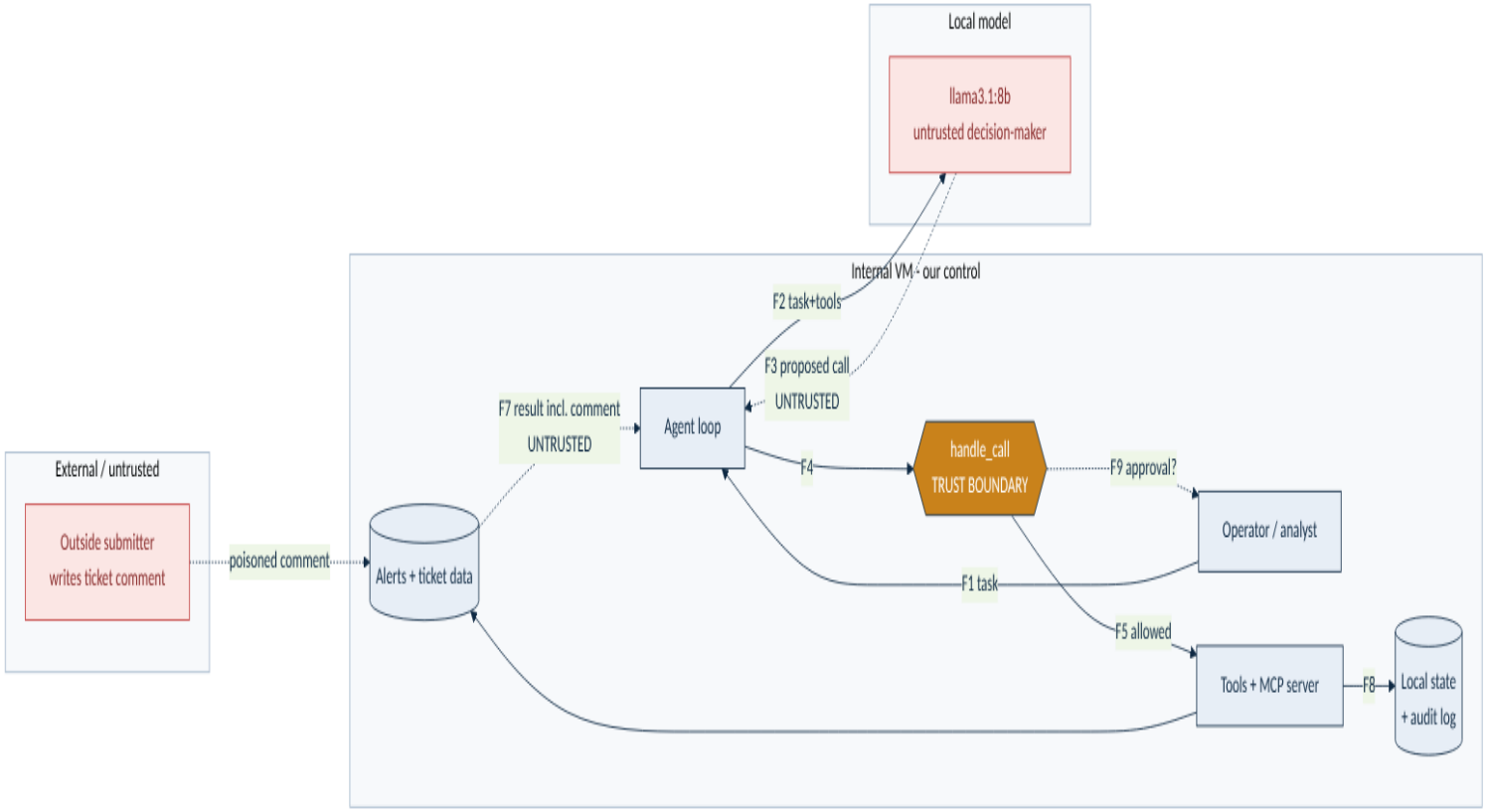


In DevOps

A deploy pipeline: untrusted input is the webhook payload and the PR contents. Same method - find the doors, decide each threat.

Data flow and trust boundaries

Where untrusted input crosses into our control is where the whole review focuses.



Untrusted input enters at:

F3 Model -> Agent
(a proposed tool call)

F7 Data -> Agent
(result incl. ticket comment)

The model is untrusted (it only proposes) and **ticket comments can be outsider-written**. Everything else stays inside our control. Find these doors first - they decide every control that follows.

Try it: tool calling



Try it: simulation

Open: **simulations/06-secure-threat-model.html**

What you will practice

1. Click the data flows you think carry attacker-controllable content, then check your answer.
2. See why only F3 (model proposes) and F7 (ticket comment) are the untrusted doors.
3. Walk the threat -> decision table: a control, or an accepted/not-applicable with a reason.
4. Answer the check questions on what a threat model establishes first.

Track B learners: do the simulation first. It takes 10 to 15 minutes and makes the lab output much easier to read.

6.2 OWASP LLM Top 10

The shared checklist for shipping an AI feature.

OWASP LLM Top 10 - coverage for this system

Each applicable risk maps to a control. The three not-applicable ones are documented, not ignored.

OWASP risk	How this system is exposed	Control
LLM01 Prompt injection	Poisoned ticket comment tries to redirect the agent	CTL-04, CTL-01
LLM06 Excessive agency	An agent with tools that disable accounts / email out	CTL-01, CTL-02, CTL-03
LLM02 Info disclosure	Private logs/CMDB could be exfiltrated	CTL-08, CTL-09
LLM05 Output handling	Acting on tool output without checking	CTL-03, CTL-04
LLM10 Unbounded use	A runaway loop exhausts the VM	CTL-05
LLM04 / LLM07 / LLM08	Poisoning / prompt-leak / vector store	N/A by design (documented)

N/A is a decision, not a gap: we never train or fine-tune (LLM04), keep no secrets in the system prompt (LLM07), and use no vector store here (LLM08). Each is recorded with its reason in the threat model.

Try it: the agent loop



Try it: simulation

Open: **simulations/06-secure-owasp.html**

What you will practice

1. Match each OWASP risk to the control that best addresses it.
2. Get feedback drawn from the real owasp.py coverage map.
3. See how documented non-applicability (LLM04/07/08) is a strength, not a gap.
4. Answer the check questions on why a shared checklist matters.

Track B learners: do the simulation first. It takes 10 to 15 minutes and makes the lab output much easier to read.

6.3 Data governance

You cannot protect - or justify - what you have not classified.

Data governance: label it before you protect it

Know, for every piece of data: how sensitive, personal or not, and where it may live.

What it is

Data governance here means a simple, honest inventory: for every data asset the system touches, record its **sensitivity** (public / internal / confidential / regulated), whether it contains **PII** (personal data), and its **residency** (where it is allowed to live). Those three facts drive everything else: sensitivity picks the controls, PII adds privacy duties, and residency decides the deployment pattern.

In other IT roles: Classification is the bridge from "it works" to "it is compliant".



Think of it like...

Labels in a hospital. A lunch menu and a patient file are not handled the same way. Each document is labelled, and the label decides who may see it and where it may go.



You've seen this before

Most data-protection findings in audits trace back to data that was never classified - so nobody knew it needed protecting or where it was allowed to go.



In the SOC

Auth logs and CMDB data are **confidential and contain PII**; ticket comments are **untrusted input**. All of it is **on-VM** - nothing leaves the network.



In DevOps

Build logs may hold secrets; classify them as confidential and keep them off public dashboards. Same discipline.

The system's data, classified

From the real system_model.py. The most sensitive asset sets the bar for the whole system.

Data asset	Sensitivity	PII	Residency
Alert metadata	internal	no	on-VM
Authentication logs	confidential	yes	on-VM
Asset owner / CMDB	confidential	yes	on-VM
Ticket comments (outsider-written)	untrusted-input	no	on-VM
Case notes + audit log	confidential	no	on-VM

Everything stays on-VM. The most sensitive data is confidential with PII - which, in 6.4, is exactly what points to hosting the model locally rather than sending anything to an outside service.

Try it: data governance



Try it: simulation

Open: **simulations/06-secure-governance.html**

What you will practice

1. Classify each of the system's real data assets by sensitivity.
2. Reveal the registered classification and see the PII and residency labels.
3. See how "confidential, on-VM" leads straight to the local-hosting decision.
4. Answer the check questions on what classification decides.

Track B learners: do the simulation first. It takes 10 to 15 minutes and makes the lab output much easier to read.

6.4 Deployment patterns by sensitivity

Where the model runs follows from the data, not from fashion.

Deployment patterns: let the data choose

Know, for every piece of data: how sensitive, personal or not, and where it may live.

What it is

There is a ladder of deployment patterns, from least to most control over your data: a **public SaaS API**, a **vendor API** with a data-protection contract and region lock, a **private-cloud / VPC-hosted** open model, and a **fully local / on-prem** open model. The rule is simple: pick the least-effort pattern whose control is **good enough for your most sensitive data**. For confidential or regulated data, that means hosting it yourself.

In other IT roles: The trade-off of local: you run and patch it, with your own capacity.



Think of it like...

Where you keep your valuables. A spare umbrella lives on the porch; your passport lives in a safe at home, not a rented locker across town. Sensitive data belongs on infrastructure you control.



You've seen this before

Teams handling regulated data increasingly run open-weight models locally precisely so that no prompt or record ever leaves their perimeter - which also simplifies the audit.



In the SOC

This system handles confidential PII, so the lab runs a **fully local open-weight model** (Ollama on the VM). No prompt, log or record leaves the network.

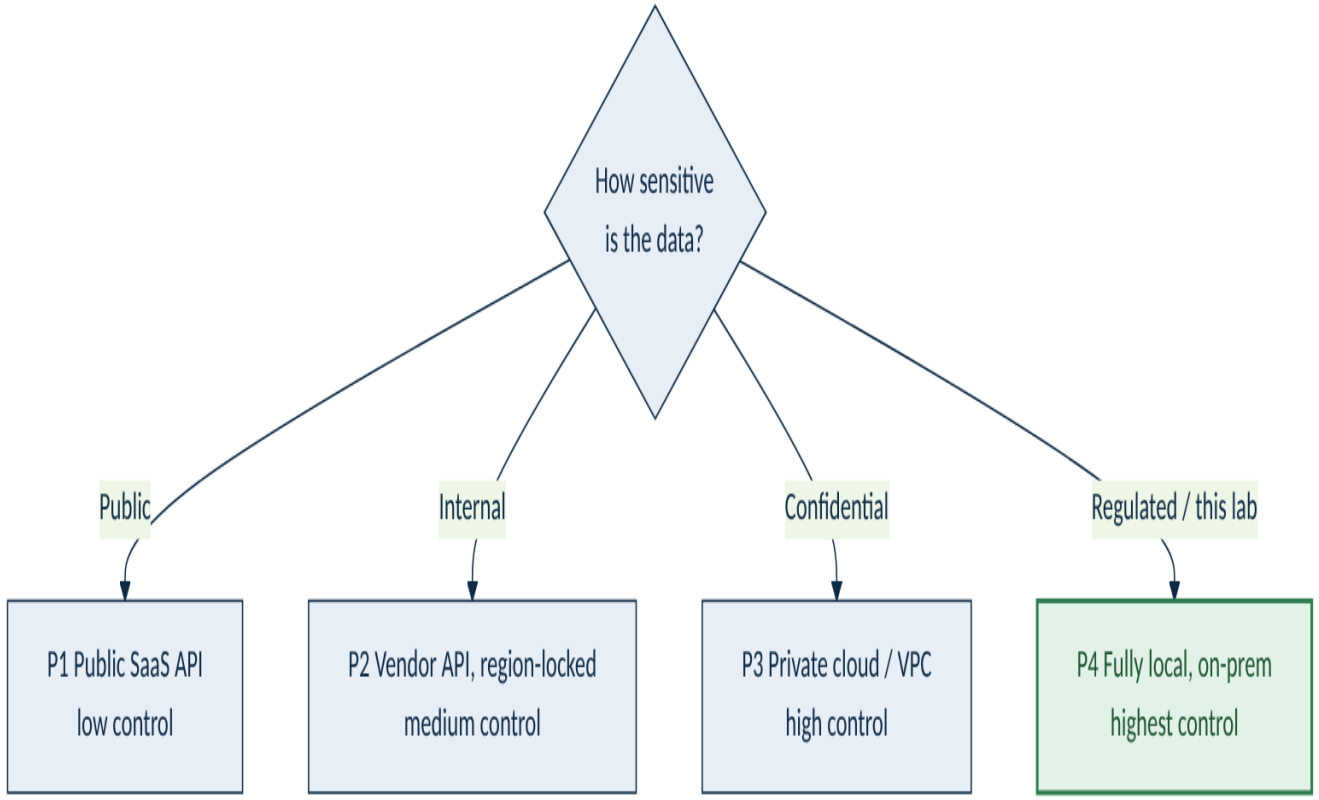


In DevOps

For a public marketing-copy helper, a SaaS API is fine. For anything touching customer data, move down the ladder toward local.

The deployment ladder - and this system's choice

Pick the least-effort pattern whose control is good enough for your most sensitive data.



This system: confidential PII

The most sensitive data is **confidential** and must not leave the network, so the recommended pattern is **P4 - fully local, on-prem** (what the lab uses).

Highest control: data never crosses the perimeter; no third-party contract to rely on; the compliance story is simply "it stays on our VM".

Trade-off: you run and patch it, and capacity is your own - fine for a CPU-only triage helper.

Try it: deployment patterns



Try it: simulation

Open: **simulations/06-secure-deployment.html**

What you will practice

1. Choose a data sensitivity and see which deployment patterns are acceptable.
2. Watch the recommendation shift toward local as sensitivity rises.
3. See why confidential data lands on fully local hosting - the real decision logic from the lab.
4. Answer the check questions on what decides the pattern and the local-hosting trade-off.

Track B learners: do the simulation first. It takes 10 to 15 minutes and makes the lab output much easier to read.

6.5 Compliance mapping

Turn engineering controls into language a CISO can sign.

The agent loop: think, act, observe, repeat

What it is

Compliance mapping records, for each control, which **framework clause** it helps satisfy and what the **evidence** is. Do it once and the same control answers many frameworks: least privilege shows up in NIST AI RMF, ISO 42001, ISO 27001 and SOC 2. The mapping becomes **defendable** only when each clause links to **evidence** - here, a passing Lab 5 test - rather than a promise.

In other IT roles: The server, not the model, decides which tools exist at all.



Think of it like...

One ingredient, many recipes. Eggs appear in the omelette, the cake and the custard. One control appears across many frameworks - map it once and reuse it.



You've seen this before

Auditors do not want prose; they want a control, the clause it meets, and proof it operates. A test that passes is the cleanest proof you can offer.



In the SOC

The ten Lab 5/6 controls map across **NIST AI RMF, ISO 42001, ISO 27001 and SOC 2**, and each clause links back to a passing test.

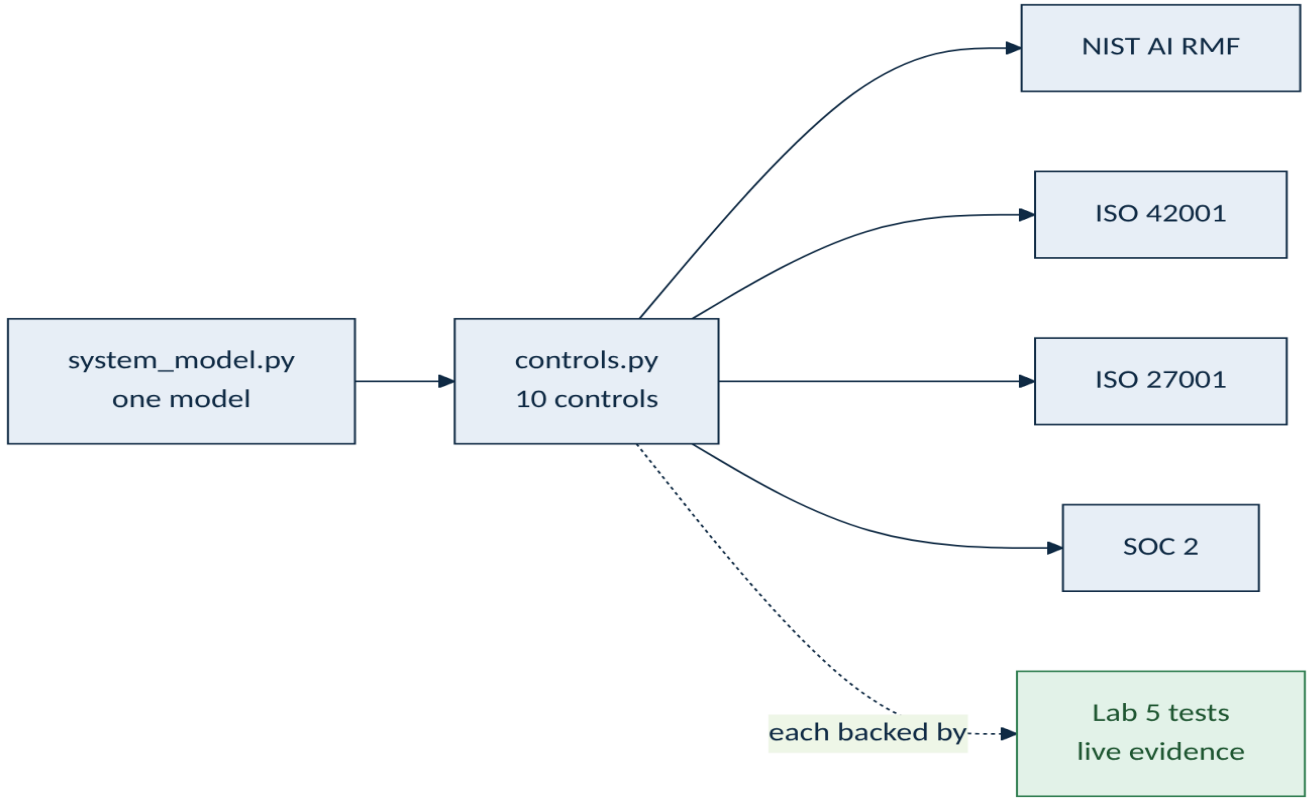


In DevOps

Wire the evidence tests into CI: if a control's test breaks, the compliance claim breaks too - and the build fails.

One model, every framework

The controls roll up to four frameworks - and each is backed by a Lab 5 test.



ıple: CTL-01 Least privilege

Framework	Clause
NIST AI RMF	MANAGE-2.2
ISO 42001	A.6.2.4
ISO 27001	A.8.2 (privileged access)
SOC 2	CC6.1 / CC6.3

Evidence: `test_triage_policy_blocks_tools_outside_the_allowlist` - a passing test, not a promise.

One control, four frameworks, one test. Map once, reuse everywhere, and let the test be the proof. Replace the example clause ids with your own framework's - the method is the transferable part.

The compliance picture at a glance

Ten controls, mapped across four frameworks - the full roll-up the lab generates.

Framework	What it is	Clauses touched
NIST AI RMF	US AI Risk Management Framework (GOVERN/MAP/MEASURE/MANAGE)	8
ISO/IEC 42001	AI Management System - Annex A controls	7
ISO/IEC 27001	Information security - Annex A controls	8
SOC 2	Trust Services Criteria (Security, Availability, Confidentiality, Privacy)	9

Every clause in these mappings links to a control, and every control to a passing test. That chain - clause -> control -> test - is what you walk a CISO or auditor through.

Try it: compliance mapping



Try it: simulation

Open: **simulations/06-secure-compliance.html**

What you will practice

1. Pick a control and see the clauses it satisfies across all four frameworks.
2. Notice how one engineering control covers several frameworks at once.
3. See the evidence test attached to each control.
4. Answer the check questions on what makes a mapping defendable.

Track B learners: do the simulation first. It takes 10 to 15 minutes and makes the lab output much easier to read.

Check yourself: sections 6.1 to 6.5



Answer each question before you move on - write it down or say it out loud. Answers are on the next slide.

1. What four questions does a **threat model** answer, and what do you find first?

2. What is the **OWASP LLM Top 10**, and why map each risk to a control (or a documented N/A)?

3. What three facts does **data governance** record for each asset, and what does each decide?

4. How does **data sensitivity** choose a **deployment pattern**? Why does confidential data point to local?

5. What makes a **compliance mapping** defendable rather than a wish-list?

Answers: sections 6.1 to 6.5

Compare with your own answers. If you missed one, re-read that part of the section before moving on.

1. What the system is, where untrusted input enters, what could go wrong, and what stops it. You find the **data flow and trust boundaries** first - you cannot protect doors you have not located.

2. The ten most important LLM-app risks - a **shared checklist** between engineers and reviewers. Mapping each to a control (or an N/A with a reason) turns "is it secure?" into ten answerable questions.

3. **Sensitivity, PII and residency.** Sensitivity picks the controls, PII adds privacy duties, and residency decides where it may be hosted.

4. Pick the least-effort pattern whose control is **good enough for the most sensitive data**. Confidential data must not leave the perimeter, so it points to **local / on-prem** hosting.

5. **Evidence.** Each control links to a passing test, so every clause is backed by proof rather than a promise.

Key takeaway: Secure AI engineering is one chain: draw the system, find the doors, decide every risk, and back each control with evidence a CISO can check.

Lab 6 - make the Lab 5 agent deployment-grade

No new agent code. You produce the document that lets you defend the agent in front of a CISO.

What you build

A **Threat Model & Control Document** generated from one machine-readable model of the system:

- data flow, trust boundaries, threats (0 open),
- OWASP + compliance mapping,
- 10 controls, each backed by a test.

What you prove

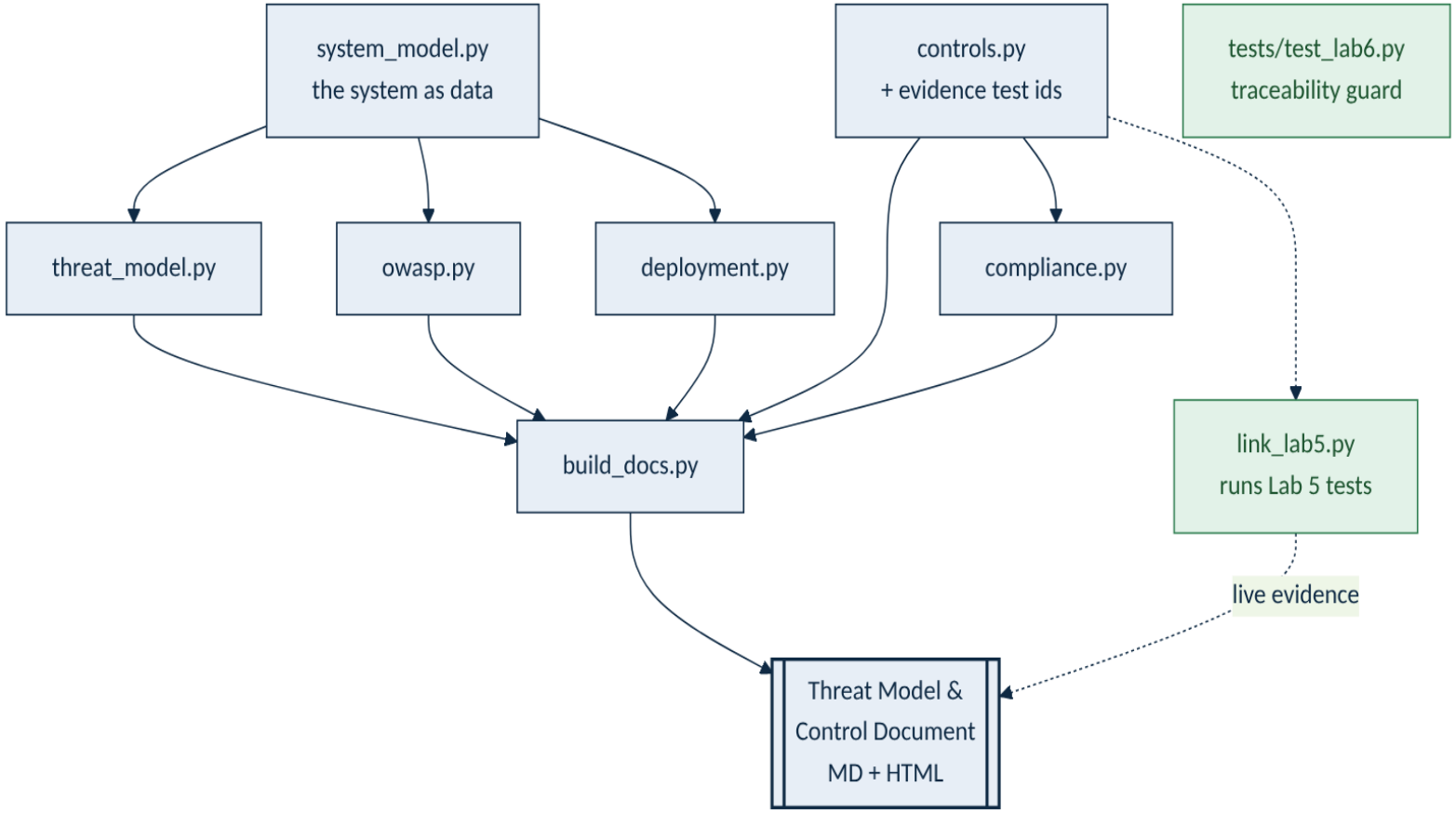
The artifacts are **complete and consistent** (18 traceability tests), and every control is backed by a **live, passing Lab 5 test**. The paperwork cannot drift from the build, and nothing is claimed without evidence.

Component	Used for	License
Python 3.9 standard library	all governance tooling + the document generator - no model, no GPU	PSF
pytest 8.4.2, bandit 1.8.6	traceability tests and the security scan	MIT, Apache 2.0
Lab 5 (beside lab6)	run live for evidence - optional; recorded evidence used if absent	from Module 5

CPU only. No model needed. All data fictional. Lab 6 reads and reasons only - it changes no system and calls no model or network. Builds directly on the Lab 5 you already secured.

How the lab fits together

Everything is generated from one model of the system - so the document can never drift from the build.



The one idea

`system_model.py` describes the system as data.

The threat model, OWASP map, deployment decision and compliance mapping are all **derived** from it.

Every control cites a **test**; `link_lab5.py` runs them for live evidence.

Change the model, regenerate - the document always matches.

Phases 1-2: setup, then the threat model

Check the environment and consistency, then derive where untrusted input enters and decide every threat.

Phase 1 –
export LAB5_DIR=/root/scripts/lab-05
python check_setup.py

```
(.venv) [root@lab-00-mary lab-06]# python check_setup.py
Lab 6 setup check

PASS  Python 3.9+ (3.9.25)
PASS  pytest installed
PASS  bandit installed
PASS  7 governance modules import
PASS  controls have evidence tests (17 distinct tests)
PASS  every threat has a decision
PASS  every applicable OWASP risk is covered (LLM04/07/08 are N/A by design)
PASS  lab-05 found for live evidence (optional) (lab-05)

All required checks passed. You are ready for Lab 6.
(.venv) [root@lab-00-mary lab-06]#
```

Phase 2 - python threat_model.py

```
(.venv) [root@lab-00-mary lab-06]# python threat_model.py
Untrusted input enters at:
  F3: Local LLM (llama3.1:8b via Ollama) -> Agent loop (agent.py)
  F4: Agent loop (agent.py) -> Trust boundary (handle_call)
  F7: Alert + ticket data (scenarios/) -> Agent loop (agent.py)

11 threats:
  T1   [Tampering/Elevation   ] Mitigated       -> CTL-01
  T2   [Elevation             ] Mitigated       -> CTL-04
  T3   [Information disclosure] Mitigated       -> CTL-08
  T4   [Elevation             ] Mitigated       -> CTL-02
  T5   [Tampering             ] Mitigated       -> CTL-03
  T6   [Denial of service     ] Mitigated       -> CTL-05
  T7   [Repudiation           ] Mitigated       -> CTL-06
  T8   [Elevation             ] Mitigated       -> CTL-07
  T9   [Information disclosure] Mitigated       -> CTL-09
  T10  [Tampering             ] Not applicable  -> Not applicable
  T11  [Information disclosure] Not applicable  -> Not applicable

validation: OK - every threat has a decision
summary: {'total': 11, 'mitigated': 9, 'not_applicable': 2, 'open': 0}
(.venv) [root@lab-00-mary lab-06]#
```

Zero open threats. Every threat maps to a control or carries a documented acceptance. That single line - "0 open" - is the first thing a reviewer looks for.

No model and no GPU are involved in Lab 6 - these commands run in under a second.

Phases 3-4: OWASP coverage and the deployment decision

Map the Top 10 to controls, then let the data choose where the model runs.

Phase 3 - python owasp.py

```
(.venv) [root@lab-00-mary lab-06]# python owasp.py
  LLM01  Prompt injection                   -> CTL-01, CTL-04, CTL-08, CTL-10
  LLM02  Sensitive information disclosure   -> CTL-06, CTL-08, CTL-09
  LLM03  Supply chain                       -> CTL-07, CTL-09
  LLM04  Data and model poisoning           -> N/A (see notes)
  LLM05  Improper output handling           -> CTL-03, CTL-04
  LLM06  Excessive agency                   -> CTL-01, CTL-02, CTL-03, CTL-07
  LLM07  System prompt leakage              -> N/A (see notes)
  LLM08  Vector and embedding weaknesses    -> N/A (see notes)
  LLM09  Misinformation                     -> CTL-10
  LLM10  Unbounded consumption              -> CTL-05

No mapped control (by design, not applicable): ['LLM04', 'LLM07', 'LLM08']
(.venv) [root@lab-00-mary lab-06]#
```

Phase 4 - python deployment.py

```
(.venv) [root@lab-00-mary lab-06]# python deployment.py
Most sensitive data in the system: confidential
Recommended deployment: P4 Fully local / on-prem open-weight model (this lab)

All patterns (most control first):
  P4  Fully local / on-prem open-weight model (this lab)    up to: regulated
  P3  Private cloud / VPC-hosted open model                 up to: confidential
  P2  Vendor API with data-protection contract (no training, region-locked)  up to: internal
  P1  Public SaaS LLM API                                   up to: public
(.venv) [root@lab-00-mary lab-06]#
```

Coverage is explicit both ways: what is addressed, and what is not applicable (with reasons). The deployment choice falls straight out of the data classification from 6.3.

Phases 5-6: compliance mapping, then build the document

Roll the controls up to the frameworks, then generate the CISO-ready document.

Phase 5 - python compliance.py

```
(.venv) [root@lab-00-mary lab-06]# python compliance.py

NIST AI RMF  (8 clauses touched)
  GOVERN-1.2           <- CTL-09
  MANAGE-1.3           <- CTL-02
  MANAGE-2.2           <- CTL-01, CTL-05, CTL-07
  MAP-1.1              <- CTL-10
  MAP-2.1              <- CTL-08
  MEASURE-2.6          <- CTL-03
  MEASURE-2.7          <- CTL-04
  MEASURE-2.8          <- CTL-06

ISO 42001  (7 clauses touched)
  A.5.2 (risk assessment) <- CTL-08
  A.6.1.2              <- CTL-10
  A.6.2.4              <- CTL-01, CTL-03, CTL-04, CTL-07
  A.6.2.6              <- CTL-05
  A.6.2.8              <- CTL-06
  A.7.2 (data management) <- CTL-09
  A.9.2 (human oversight) <- CTL-02
```

```
ISO 27001  (8 clauses touched)
  A.5.3                <- CTL-02
  A.5.34 (privacy)     <- CTL-09
  A.8.12 (data leakage prevention) <- CTL-08
  A.8.15 (logging)     <- CTL-06
  A.8.2 (privileged access) <- CTL-01, CTL-07
  A.8.25               <- CTL-10
  A.8.28 (secure coding) <- CTL-03, CTL-04
  A.8.6 (capacity)     <- CTL-05

SOC 2  (9 clauses touched)
  A1.1                 <- CTL-05
  C1.1 / P4.1          <- CTL-09
  CC6.1                <- CTL-02
  CC6.1 / CC6.3        <- CTL-01
  CC6.3                <- CTL-07
  CC6.7                <- CTL-08
  CC6.8                <- CTL-04
  CC7.2                <- CTL-06
  CC8.1                <- CTL-03, CTL-10
(.venv) [root@lab-00-mary lab-06]#
```

One command regenerates the whole document - Markdown for the repo and HTML to read or print. Re-run it any time the system model changes and the paperwork updates itself.

Phases 5-6: compliance mapping, then build the document

Roll the controls up to the frameworks, then generate the CISO-ready document.

Phase 6 - python build_docs.py

```
(.venv) [root@lab-00-mary lab-06]# python build_docs.py
Wrote:
  /root/scripts/lab-06/out/Threat_Model_and_Control_Document.md
  /root/scripts/lab-06/out/Threat_Model_and_Control_Document.html

10 controls | 11 threats (9 mitigated, 0 open) | 17 evidence tests cited
(.venv) [root@lab-00-mary lab-06]#
```

One command regenerates the whole document - Markdown for the repo and HTML to read or print. Re-run it any time the system model changes and the paperwork updates itself.

What the generated document contains

A single, regenerable document a CISO can sign - nine sections, every claim evidenced.

1-2 System + data flow

Purpose, components, data flows, trust boundaries, and where untrusted input enters.

5.2 The agent loop

Think, act, observe, repeat - until an answer or a hard limit. The loop must be able to stop.

5.3 MCP

A standard plug for tools. Expose only what is needed - least privilege at the server.

5.4 When not to

Fixed steps? Use a script or RAG. No model in the loop, nothing to hijack.

5.5 Security

Break the lethal trifecta; five defences in code; one trust boundary.

5.6 Lab 5

Built an agent, hijacked v1, defended v2. Proven with 24 tests and a clean scan.

The one sentence to remember: an agent's power is its freedom to choose its own steps - so its safety must live in code at the one point where any tool actually runs.

Phase 7: live evidence - the claims are real

Every control cites a Lab 5 test. This runs them, so the document is backed by proof, not promises.

```
$ python link_lab-05.py
```

```
(.venv) [root@lab-00-mary lab-06]# python link_lab-05.py
Running Lab 5 evidence tests in: /root/scripts/lab-05
24 passed in 0.11s
LIVE EVIDENCE: all cited controls are backed by passing tests
(.venv) [root@lab-00-mary lab-06]#
```

Why this is the key move

A control that claims "least privilege" means nothing until a test proves a forbidden tool is actually refused. Lab 6 cites that test; link_lab5 runs it. The chain is clause -> control -> **passing test**.

Works with or without Lab 5

Put lab-05 beside lab-06 and the evidence runs live. If it is absent, the document still cites the tests by name as recorded evidence - nothing in Lab 6 breaks.

Phase 8: verify - the traceability guard

The lab IS the traceability. These tests stop the document shipping with a hole.

```
$ python link_lab-05.py
```

```
(.venv) [root@lab-00-mary lab-06]# python link_lab-05.py
Running Lab 5 evidence tests in: /root/scripts/lab-05
24 passed in 0.11s
LIVE EVIDENCE: all cited controls are backed by passing tests
(.venv) [root@lab-00-mary lab-06]#
```

Why this is the key move

A control that claims "least privilege" means nothing until a test proves a forbidden tool is actually refused. Lab 6 cites that test; link_lab5 runs it. The chain is clause -> control -> **passing test**.

Works with or without Lab 5

Put lab-05 beside lab-06 and the evidence runs live. If it is absent, the document still cites the tests by name as recorded evidence - nothing in Lab 6 breaks.

Defending it in front of the CISO

The document is built to answer the questions a reviewer actually asks - here they are, with where each is answered.

The CISO asks...	Your answer (and where it is in the document)
"Where can untrusted input get in?"	Section 2: two trust-boundary crossings (the model, and ticket comments) - drawn and listed.
"What could go wrong, and did you miss anything?"	Section 3: STRIDE over every boundary; 11 threats, 0 open; N/A items have reasons.
"Are you covered for the known LLM risks?"	Section 4: OWASP LLM Top 10, each mapped to a control or a documented N/A.
"Where does our data go?"	Sections 6 + 8: fully local hosting; every asset on-VM; nothing leaves the network.
"Prove the controls actually work."	Section 5 + link_lab5: each control is backed by a passing test, run live.
"How does this map to our framework?"	Section 7: NIST AI RMF, ISO 42001, ISO 27001, SOC 2 - swap clause ids for yours.

You are never improvising. Every likely question has a section and an evidenced answer - which is exactly what "defendable" means.

Lab 6 - what you produced

A working agent is not a deployable one. Now it is both - and you can prove it.

Before Module 6

A defended agent (Lab 5) that works and resists a hijack - but with:

- no written threat model,
- no OWASP mapping,
- no compliance story,
- nothing a CISO could sign.

Good engineering, not yet deployable.

After Module 6

The same agent, now with:

- a threat model, 0 open,
- OWASP LLM Top 10 coverage,
- data governance + local hosting,
- compliance across 4 frameworks,
- every control backed by a test.

Deployable, and defendable.

The lesson

Security is not just the code - it is the **evidence** that the code does what you claim.

Generate the document from one model, back every control with a test, and the paperwork stays true to the build.

That is secure AI engineering.

Stretch (extra/STRETCH.md): add a component and watch a new threat appear; map to your own framework; break the traceability on purpose and watch a test catch it.

How Module 6 builds on Modules 0 to 5

This module is the capstone: it wraps everything you built into something a CISO can sign.

From	What it gave you	Where it shows up in Module 6
Module 0	The VM and local model (Ollama)	The hosting decision (6.4): fully local, nothing leaves the network.
Module 1	Safety rules and "assume input is hostile"	The mindset behind the threat model and the trust boundaries (6.1).
Module 2	Automated tests and the Bandit scan	The evidence: every control cites a test; 18 traceability tests guard the document.
Module 3	Tool calling and <tags>	The components and data flows being threat-modelled; output-as-data as a control.
Module 4	RAG and the audit-log habit	Data governance (6.3) and the audit control (CTL-06).
Module 5	The built-and-defended agent	The exact system Lab 6 documents - its five defences become the control catalogue.

At work: run any new AI automation through this chain before go-live - threat model, OWASP, governance, deployment, compliance, evidence. Generate the document from one model so it never goes stale.

Module 6 complete

You threat-modelled an AI system, mapped it to the OWASP LLM Top 10, governed its data, chose a local deployment, and mapped every control to NIST, ISO and SOC 2 - each backed by a passing test. The result is a document a CISO can sign, generated from one model so it never drifts from the build.

Before you go: take the final check (simulations/06-secure-final-assessment.html), and try the stretch tasks in extra/STRETCH.md - add a component and watch a new threat appear, then defend against it.